

Java Core & OOPs Backend

Complete Handcrafted Interview Revision Blueprint

Comprehensive Revision Study Companion

Compiled on June 11, 2026

Contents

1	Core Object-Oriented Programming (OOP) Pillars	3
1.1	1. Class	3
1.2	2. Methods	3
1.3	3. Inheritance	4
1.4	4. Polymorphism	4
2	Detailed Deep Dive: Polymorphism & Method Dispatch	6
2.1	1. Method Overloading (Compile-Time Polymorphism)	6
2.2	2. Method Overriding (Runtime Polymorphism)	6
3	Encapsulation & Abstraction Blueprint	8
3.1	1. Encapsulation (Data Hiding)	8
3.2	2. Abstraction	8
4	Constructors, Contexts & Access Control	10
4.1	1. Constructors	10
4.2	2. Access Modifiers	10
4.3	3. Keywords: this, super, and final	10
4.4	4. The Universal Object Class	11
4.5	5. Inner Classes	11
5	Software Design: Coupling & Cohesion	12
5.1	1. Definitions	12
5.2	2. Achieving High Cohesion & Low Coupling	12
6	Interfaces, Abstract Classes & Why Instantiation Fails	14
6.1	1. Abstract Classes	14
6.2	2. Interfaces	14
7	Exception Handling Architecture	16
7.1	1. Core Exception Concept	16
7.2	2. The Complete Exception Hierarchy	16
7.3	3. Checked vs. Unchecked Exceptions	16
7.3.1	A. Checked Exceptions (The Strict Supervisor)	16
7.3.2	B. Unchecked Exceptions (The Runtime Surprise)	17

7.4	4. Action vs. Declaration: throw vs. throws	17
7.5	5. Exception Propagation	18
7.6	6. Custom Exceptions (Subclassing Exception)	19

1 Core Object-Oriented Programming (OOP) Pillars

1.1 1. Class

- **What is it:** A class is a blueprint used to create objects. It defines properties (variables) and behaviors (methods) of an entity.
- **Benefit:** It helps organize code logically and promotes reusability. It makes large programs easier to maintain.
- **When to use:** Use a class when representing real-world entities like `Student` or `Employee`. It is used whenever structured data is needed.

```
class Student {  
    String name;  
    int marks;  
}
```

Listing 1: Class Implementation Example

How It Works Internally

The class defines structural rules, but memory is only allocated when objects are created. The JVM uses class definitions loaded in memory to instantiate objects on the Heap at runtime.

Practical Scenario

Used in enterprise school management systems where unique student objects (e.g., Alice, Bob) are created dynamically to hold grade-book details.

1.2 2. Methods

- **What is it:** A method is a block of code that performs a specific task. It defines the behavior of an object.
- **Benefit:** Reduces code duplication and improves readability. Makes debugging easier.
- **When to use:** Use methods when the same operational logic needs to be repeated multiple times with different values.

```
class Calculator {  
    void add(int a, int b) {  
        System.out.println(a + b);  
    }  
}
```

Listing 2: Method Implementation Example

How It Works Internally

Method calls utilize Thread Stack Memory. Each call instantiates a new stack frame containing local reference variables and arguments. The execution thread pops and discards the stack frame immediately upon method completion.

1.3 3. Inheritance

- **What is it:** Inheritance allows one class to acquire properties and behaviors of another class. It creates a parent-child relationship.
- **Benefit:** Promotes code reuse and reduces logical redundancy.
- **When to use:** Use when there is an explicit "IS-A" relationship between entities (e.g., Dog is an Animal).

```
class Animal {  
    void eat() {  
        System.out.println("Eating");  
    }  
}  
  
class Dog extends Animal {  
    // Automatically acquires the eat() behavior  
}
```

Listing 3: Inheritance Implementation Example

How It Works Internally

The child class reuses the parent's bytecode patterns. The JVM links classes using the `extends` keyword, allowing instance lookups to check child and parent method tables at execution time.

1.4 4. Polymorphism

- **What is it:** Polymorphism allows one method to behave differently based on the underlying object executing it. It occurs through overloading and overriding.
- **Benefit:** Improves system flexibility, maintainability, and scalability.
- **When to use:** Use when the same method name should yield different behaviors based on the active child instance.

```
// Parent class  
class Payment {  
    void makePayment() {  
        System.out.println("Processing payment...");  
    }  
}  
  
// Child class for Credit Card payment  
class CreditCardPayment extends Payment {  
    @Override  
    void makePayment() {  
        System.out.println("Payment made using Credit Card");  
    }  
}  
  
// Child class for UPI payment  
class UPIPayment extends Payment {
```

```

    @Override
    void makePayment() {
        System.out.println("Payment made using UPI");
    }
}

// Main class
public class MainApp {
    public static void main(String[] args) {
        Payment p;

        // Assigning CreditCardPayment object
        p = new CreditCardPayment();
        p.makePayment();

        // Assigning UPIPayment object
        p = new UPIPayment();
        p.makePayment();
    }
}

```

Listing 4: Polymorphic Payment Flow Example

2 Detailed Deep Dive: Polymorphism & Method Dispatch

2.1 1. Method Overloading (Compile-Time Polymorphism)

- **What is it:** Having multiple methods with the same name but different parameters (either different parameter types, numbers, or positions) inside a single class. Resolves during compilation.
- **Benefit:** Reduces duplication of method names, makes the codebase easier to read, increases flexibility, and helps reuse similar logic across differing inputs. No inheritance required.
- **When to use:** When a similar operation must be performed with variable kinds of inputs (e.g., mathematical operations or utility helper routines).

```
class BankSystem {  
    // Method 1: Takes ONE account parameter (Internal Transfer)  
    void transfer(int accountNo, double amount) {  
        System.out.println("Transferring INR " + amount + " internally to  
Account: " + accountNo);  
    }  
  
    // Method 2: Same name, but takes THREE parameters (External Transfer)  
    void transfer(int accountNo, String ifscCode, double amount) {  
        System.out.println("Transferring INR " + amount + " externally to  
Account: " + accountNo + " via IFSC: " + ifscCode);  
    }  
}
```

Listing 5: Method Overloading Example

How It Works Internally

The compiler inspects arguments passed to the method during compilation, matching arguments to the method signatures. It binds the method call directly to the bytecode representation. This is why it is faster than overriding.

Practical Scenario

Frequently utilized in Calculator applications where `add(int, int)` and `add(double, double)` coexist cleanly under a single descriptive name.

2.2 2. Method Overriding (Runtime Polymorphism)

- **What is it:** Overriding occurs when a child class provides its own customized implementation of a parent method. It must match the exact name, parameter signature, and return types.
- **Benefit:** Allows changing behavior without modifying the parent class. Supports loose coupling and easy application scalability.
- **When to use:** When a child class requires uniquely distinct behavior compared to its parent template.

```
class Payment {  
    void pay() {  
        System.out.println("Processing payment");  
    }  
}  
  
class CreditCard extends Payment {  
    @Override  
    void pay() {  
        System.out.println("Paid using Credit Card");  
    }  
}
```

Listing 6: Method Overriding Example

How It Works Internally

The JVM resolves the method target at runtime using ****Dynamic Method Dispatch****. A parent class reference variable stores a child object, but method execution calls the child's overridden code block. This introduces a minor runtime lookup overhead compared to compile-time overloading.

Practical Scenario

Essential for dynamic systems like Unified Payment Interfaces where different pathways (UPI, Card, NetBanking) execute unique code blocks via a single shared interface template.

3 Encapsulation & Abstraction Blueprint

3.1 1. Encapsulation (Data Hiding)

- **What is it:** The process of binding data variables and methods together into a single cohesive unit (a class) while restricting direct public access to prevent logical corruption.
- **Benefit:** Boosts security by isolating data. Allows updating validation boundaries seamlessly without breaking outside consumer code.
- **When to use:** Always declare state fields as `private` and expose validation controls selectively using public getters and setters.

Without Encapsulation (Vulnerable State Modification)

```
class BankAccount {
    public double balance = 500.00; // Public field exposes risk
}

public class Main {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();
        account.balance = -999999; // State is corrupted with zero
        validations!
    }
}
```

```
class BankAccount {
    // 1. Data hidden securely from direct access
    private double balance = 500.00;

    // 2. Getter: Controls data read access
    public double getBalance() {
        return balance;
    }

    // 3. Setter: Controls modification with boundary rules
    public void deposit(double amount) {
        if (amount > 0) { // Safety rule
            balance += amount;
        } else {
            System.out.println("Invalid deposit amount!");
        }
    }
}
```

Listing 7: Encapsulation Applied Safely

3.2 2. Abstraction

- **What is it:** Hiding complex internal implementation details from the user, exposing only clean, essential functionality. Focuses on **what** the system does rather than **how** it is built.

- **Benefit:** Greatly reduces client-facing system complexity and isolates consumer dependencies.
- **When to use:** Use when defining generic architectures with custom underlying operational pathways.

```
// Abstract class: Cannot be instantiated directly
abstract class Vehicle {
    // Abstract method: Has NO body
    abstract void accelerate();

    // Concrete method: Sharing common helper logic
    void turnOnHeadlights() {
        System.out.println("Headlights turned on.");
    }
}

class PetrolCar extends Vehicle {
    @Override
    void accelerate() {
        System.out.println("Injecting petrol... Pistons moving... Speeding up!");
    }
}

class ElectricCar extends Vehicle {
    @Override
    void accelerate() {
        System.out.println("Drawing current... Motor spinning... Speeding up quietly!");
    }
}
```

Listing 8: Abstraction Implementation

4 Constructors, Contexts & Access Control

4.1 1. Constructors

- **What is it:** A specialized class method executed automatically when an object is created with the `new` keyword. It shares the exact name of the class and does not declare any return type.
- **Benefit:** Guarantees that objects are fully initialized with necessary states before application usage, reducing null references and bugs.
- **When to use:** Use to enforce parameter requirements during object instantiation.

How It Works Internally

When `new` is called, the JVM allocates heap space and executes the constructor method block. If no constructor is defined in the source code, the Java compiler injects a default, no-argument constructor implicitly.

4.2 2. Access Modifiers

- **What is it:** Accessibility constraints written to define package-level security.
- **Benefit:** Secures code logic, reinforces encapsulation boundaries, and prevents illegal state mutations.

Table 1: Access Modifier Visibility Matrix

Modifier	Same Class	Same Package	Subclass (Inheritance)	Global World
private	Yes	No	No	No
default	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

4.3 3. Keywords: `this`, `super`, and `final`

- **`this`:** Refers directly to the current object. Helps resolve namespace conflicts between local parameters and class fields.
- **`super`:** Reference pointing directly to the parent class object. Triggers parent constructors or invokes overridden methods.
- **`final`:** Imposes strict immutability. Applied to variables, they become constant; to methods, they prevent overriding; to classes, they stop inheritance.

```
class Parent {
    Parent() {
        System.out.println("Parent initialized.");
    }
}

class Child extends Parent {
```

```

String name;

Child(String name) {
    super(); // Calls parent constructor
    this.name = name; // Differentiates local parameter
}
}

```

Listing 9: Constructors and Context Keywords Example

4.4 4. The Universal Object Class

- **What is it:** The `java.lang.Object` class is the ultimate parent of all classes.
- **Benefit:** Standardizes fundamental APIs like `toString()`, `equals()`, and `hashCode()` across all runtime types.

4.5 5. Inner Classes

- **What is it:** A nested class structure built inside another class body.
- **Benefit:** Tightens logical groupings and allows inner classes to access outer private variables.
- **When to use:** Used heavily in GUI frameworks, button event handling, and thread operations.

```

class Outer {
    class Inner {
        void display() { System.out.println("Inner execution."); }
    }
}
// Outer outer = new Outer();
// Outer.Inner inner = outer.new Inner();

```

Listing 10: Inner Class Instantiation

5 Software Design: Coupling & Cohesion

5.1 1. Definitions

- **Coupling:** The strength of mutual dependence between different classes. High coupling creates fragile ecosystems where updating one module breaks another. Aim for ****Low Coupling****.
- **Cohesion:** The single-minded focus of class duties. Low cohesion occurs when a class executes unrelated tasks. Aim for ****High Cohesion****.

Bad Design (High Coupling, Low Cohesion)

```
// One class doing everything - Highly Cohesive Fail!
class FoodApp {
    void createOrder() {
        System.out.println("Order created");
    }
    void makePayment() {
        System.out.println("Payment done");
    }
    void assignDelivery() {
        System.out.println("Delivery assigned");
    }
}
```

Listing 11: Monolithic God Class Example

The Critical Problem: If payment gateway security policies shift, developers must modify and redeploy this unified FoodApp class, risking unexpected regressions in delivery and order modules.

5.2 2. Achieving High Cohesion & Low Coupling

By decomposing operations into small, single-purpose classes, we guarantee clean isolation.

```
class OrderService {
    void createOrder() { System.out.println("Order Processed"); }
}

class PaymentService {
    void executePayment() { System.out.println("Payment Processed"); }
}

class DeliveryService {
    void assignCourier() { System.out.println("Courier Dispatched"); }
}
```

Listing 12: Decoupled Cohesive Architecture

Benefit & Value

Enables development teams to operate independently, streamlines automated testing, and allows microservice integrations.

Practical Scenario

In production banking and e-commerce applications, payment engines can be upgraded or hot-swapped without touching order processing databases or courier systems.

One-Line Interview Answer

"Low coupling reduces dependency between classes, and high cohesion ensures each class has a single responsibility, making the system flexible and maintainable."

6 Interfaces, Abstract Classes & Why Instantiation Fails

6.1 1. Abstract Classes

- **What is it:** A conceptually incomplete class declared with the `abstract` keyword. It can house abstract (bodyless) and concrete methods.
- **Benefit:** Allows sharing common fallback implementations across subclasses while enforcing strict method overrides.
- **When to use:** When classes share generic behavioral patterns but require specialized overrides for specific methods.

```
abstract class Payment {
    abstract void makePayment();

    void display() {
        System.out.println("Payment processing system active.");
    }
}

class CreditCard extends Payment {
    void makePayment() {
        System.out.println("Paid using Credit Card");
    }
}
```

Listing 13: Abstract Class Payment Workflow

Why Abstract Classes Cannot Be Instantiated

"An abstract class cannot be instantiated because it is **conceptually incomplete**. It acts as a strict structural blueprint, containing abstract methods without implementation bodies. If Java allowed you to create an object of an abstract class and call an empty, bodyless method, the JVM wouldn't know what instructions to run, causing the program to crash. Therefore, Java restricts instantiation at compile-time to guarantee safety and clean design."

6.2 2. Interfaces

- **What is it:** A contract structure containing only abstract method definitions (prior to Java 8 default/static methods). Supports total abstraction.
- **Benefit:** Promotes extreme loose coupling and supports multiple inheritance pathways.
- **When to use:** When implementing generic shared APIs across completely unrelated class domains.

```
interface PaymentGateway {
    void processTransaction();
}

class UPIPayment implements PaymentGateway {
```

```
public void processTransaction() {  
    System.out.println("Paid using UPI gateway.");  
}  
}
```

Listing 14: Interface Contract Pattern

7 Exception Handling Architecture

7.1 1. Core Exception Concept

- **What is it:** Exception handling is a structured framework to capture and resolve runtime errors, preventing program termination.
- **Benefit:** Prevents sudden server crashes, ensures a clean user experience, and simplifies debugging through structured error handling.
- **When to use:** Essential when executing unreliable interactions with external components like networks, databases, or disks.

```
try {  
    int a = 10 / 0; // Throws ArithmeticException  
} catch (ArithmeticException e) {  
    System.out.println("Gracefully handled error: " + e.getMessage());  
}
```

Listing 15: Try-Catch Operational Flow

How It Works Internally

If an operational failure is encountered, the JVM instantiates a matching Exception object. The JVM then traverses backwards through the Thread Call Stack to find a compatible catch handler. If none is found, the thread is abruptly terminated.

7.2 2. The Complete Exception Hierarchy

Below is the standard, compiler-enforced Java Throwable hierarchy:

Throwable Class Hierarchy

- **Throwable** (The parent class of all errors)
 - **Error** (Severe system faults - do not attempt to handle)
 - ✱ OutOfMemoryError, StackOverflowError, VirtualMachineError
 - **Exception** (Recoverable application issues)
 - ✱ **Checked Exceptions** (Compile-time): IOException, SQLException, FileNotFoundException
 - ✱ **Unchecked Exceptions** (Runtime): RuntimeException and its subclasses (NullPointerException, ArithmeticException, ArrayIndexOutOfBoundsException, IllegalArgumentException)

7.3 3. Checked vs. Unchecked Exceptions

7.3.1 A. Checked Exceptions (The Strict Supervisor)

These are structural risks that the Java compiler forces you to handle using a structured try-catch block or declare in the signature via throws before compiling.


```
import java.io.*;

class Test {
    public static void main(String[] args) {
        // ERROR: Unhandled exception type FileNotFoundException
        FileReader file = new FileReader("C:\\notes.txt");
    }
}
```

Listing 16: Checked Exception Compile Failure

```
try {
    FileReader file = new FileReader("C:\\notes.txt");
} catch (FileNotFoundException e) {
    System.out.println("File missing on local drive!");
}
```

Listing 17: Checked Exception Fix

7.3.2 B. Unchecked Exceptions (The Runtime Surprise)

The compiler ignores these during build time. They occur at runtime and are typically caused by flaws in the developer's application logic.

```
class Test {
    public static void main(String[] args) {
        // Compiles cleanly, but crashes at runtime with ArithmeticException
        int result = 10 / 0;
    }
}
```

Listing 18: Unchecked Exception Code

```
int divisor = 0;
if (divisor != 0) {
    int result = 10 / divisor;
} else {
    System.out.println("Cannot divide by zero!");
}
```

Listing 19: Defensive Logic Fix (No heavy try-catch required)

7.4 4. Action vs. Declaration: throw vs. throws

- **throw:** Used inside a method to explicitly throw an exception object immediately.
- **throws:** Written in the method signature to warn callers that this method may throw exceptions.

Table 2: throw vs. throws Comparison

Feature	throw	throws
Purpose	Directly throws an exception instance.	Declares potential exceptions.
Placement	Inside the method body.	In the method signature.
Followed by	An exception instance object.	Exception class names.
Count	Only one exception instance.	Can declare multiple comma-separated classes.

```
public void test() throws IOException {
    throw new IOException("Error occurred");
}
```

Listing 20: Combining throw and throws

Why do we need throws if we already used throw? Because `IOException` is a Checked exception. The compiler strictly mandates that you must either catch it locally or declare it in the method signature using `throws` to pass the responsibility to the caller. Without the `throws` keyword in the signature, the code will fail to compile.

7.5 5. Exception Propagation

Exception propagation means an unhandled exception automatically bubbles up the call stack until it is handled.

```
import java.io.IOException;

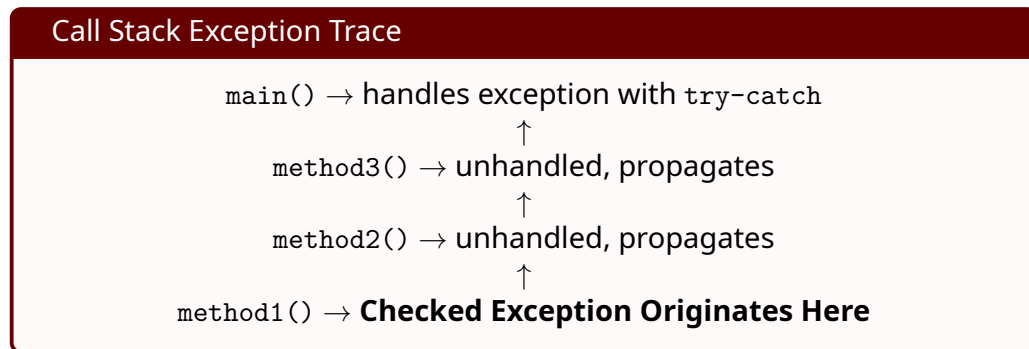
class PropagationDemo {
    // Exception originates here
    public static void method1() throws IOException {
        throw new IOException("Error in method1");
    }

    // Method 2 calls method1, propagating the exception upward
    public static void method2() throws IOException {
        method1();
    }

    // Method 3 propagates it again
    public static void method3() throws IOException {
        method2();
    }

    // Handled at the top level in main()
    public static void main(String[] args) {
        try {
            method3();
        } catch (IOException e) {
            System.out.println("Handled in main: " + e.getMessage());
        }
    }
}
```

Listing 21: Exception Propagation Flow



7.6 6. Custom Exceptions (Subclassing Exception)

Create application-specific exceptions by extending the standard `Exception` class.

```
// Custom checked exception class
class InvalidAgeException extends Exception {
    InvalidAgeException(String msg) {
        super(msg); // Passes custom error message to parent Exception class
    }
}

public class MainApp {
    public static void main(String[] args) {
        int age = 15;
        if (age < 18) {
            try {
                throw new InvalidAgeException("Invalid registration age");
            } catch (InvalidAgeException e) {
                System.out.println("Caught exception: " + e.getMessage());
            }
        }
    }
}
```

Listing 22: Custom Exception Implementation